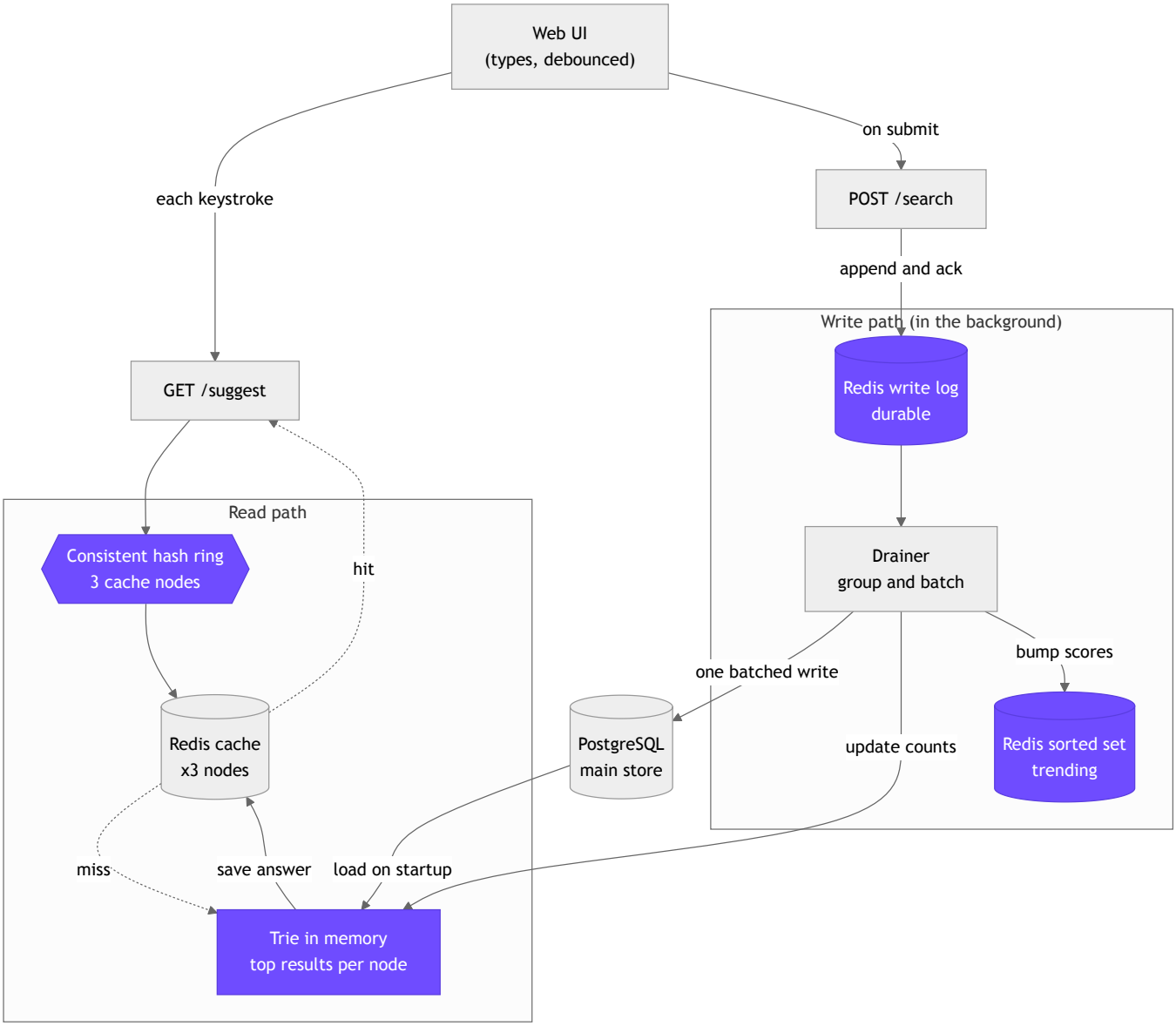
# Search Typeahead System

## 1. What this project is

This is a search box that suggests popular searches as you type. You enter a few letters and it shows up to ten matches, with the most searched ones on top. Pressing enter records that search so the list stays fresh, and a small trending panel shows what is busy right now. The backend is TypeScript with Fastify. Three Redis servers act as a shared cache, PostgreSQL is the main store, and the front end is a small Next.js app.

## 2. Architecture




*The read path and the write path. The four boxes in purple are where the main ideas live.*

Reads run on every keystroke and writes run on submit. Reads happen far more often, so the design makes reads cheap and pushes the writing into the background.

### The read path

When you type, the request checks the cache first. The cache is split across three Redis servers. To decide which server holds a given prefix I use consistent hashing, so one prefix always maps to the same server. If that server already has the answer, it comes straight back. This is the common case and it is very fast.

If the cache does not have it, the server asks the trie. The trie is the search index and it sits in memory. Every node in the trie already holds the best results for the words below it. So answering a prefix is just walking the letters of that prefix and reading the list that is waiting there. There is no scanning of the whole branch. The answer is then saved into the cache so the next person gets a hit.

### The write path

When you submit a search the server does one quick step. It adds the search to a list in Redis and replies right away with a short message. The real counting is done in the background. A small worker reads that list in batches, adds up the repeats, and writes them to PostgreSQL in a single statement. The same batch also updates the trie in memory and bumps the trending scores.

Because the list lives in Redis, and Redis writes to disk, nothing is lost if the server crashes. On restart the worker simply picks up the list again and finishes the job. This is the part I am most happy with, since a plain in memory buffer would lose its last batch on a crash.

## 3. Dataset and how to load it

The app needs a list of searches together with how many times each one was searched. The format is just a query and a count.

For ease of setup the loader can build its own data. It makes around 150,000 searches where a few are very popular and most are rare, which is how real search traffic looks. This needs no download and it meets the requirement of at least 100,000 searches.

```
docker compose up -d                       # start postgres and 3 redis
pnpm install
pnpm --filter @ta/server load              # ~150k searches, no download
```

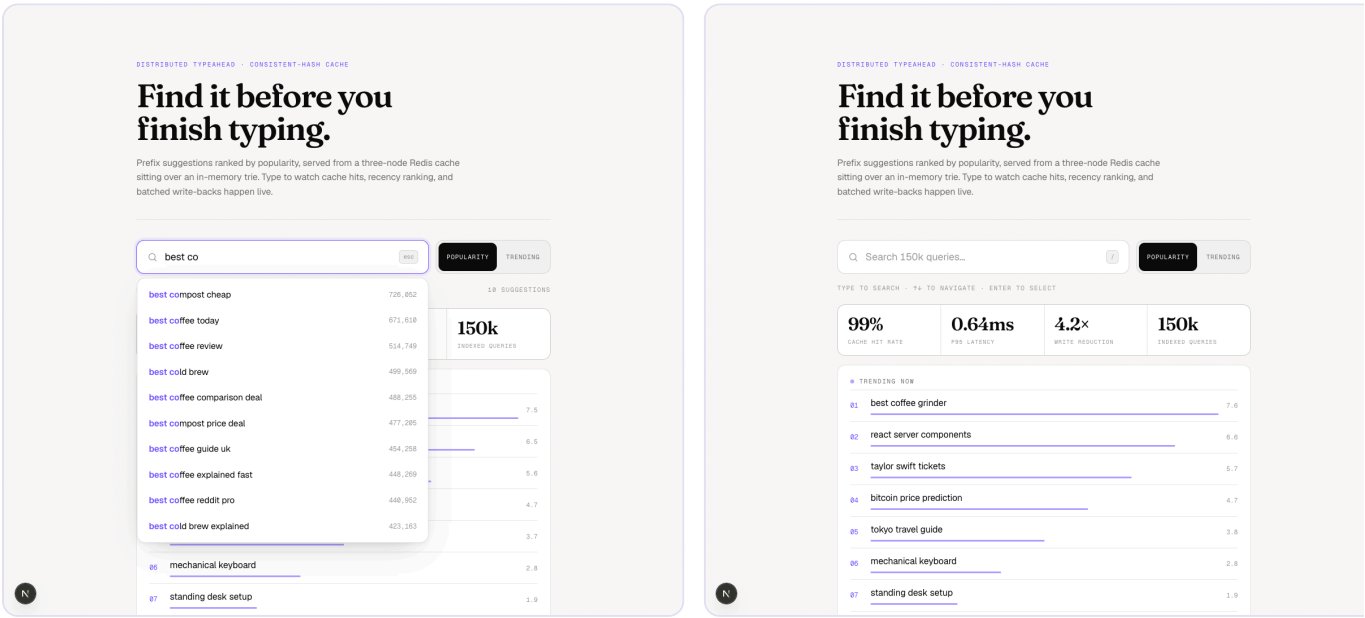
It can also read a real search log. It opens the file, counts how often each search appears, and loads those counts.

```
pnpm --filter @ta/server load --file queries.csv --top 1000000 --min-count 2
```

## 4. API

| Method | Endpoint | What it does |
|---|---|---|
| GET | `/suggest?q=<prefix>&mode=count\|recency` | Up to 10 matches that start with the prefix, sorted by count. `count` ranks by all time popularity, `recency` mixes in how recent the searches are. |
| POST | `/search` `{"query":"..."}` | Records the search and replies `{"message":"Searched"}`. The count is saved in the background. |
| GET | `/trending?n=10` | The current trending list, which fades over time. |
| GET | `/cache/debug?prefix=<p>` | Shows which cache server owns the prefix and whether it is a hit or a miss right now. |
| GET | `/cache/ring?sample=N` | Shows how a sample of keys spreads across the cache servers. |
| GET | `/metrics` | Cache hit rate, database write counts and reduction, and p50, p95, p99 latency. |

A quick example. Typing `best co` returns matches like `best coffee grinder` with their counts, and tells you it came from the cache and which server answered.




*The live UI. Each result shows whether it came from the cache or the trie. The metrics and the trending list update as you type.*

## 5. Design choices and trade offs

### A trie that already holds the top results at each node

A trie lets me find every search that starts with a prefix by walking the letters of that prefix. The twist is that each node also keeps the best results for everything below it. So a lookup never scans a whole branch. The reason this stays cheap is that counts only go up. When a count rises I walk that one search up its own path

and slot it into the lists along the way. The cost is memory, since every node keeps a small list. At this size that is fine. If the list grew into the millions I would only keep these lists for short, busy prefixes and fall back to a small walk for deep ones.

### Consistent hashing instead of a plain remainder

The cache is shared across three servers, so I need a rule that always sends one prefix to the same server. A simple remainder of the hash would work until the number of servers changes, and then almost every key moves to a new server and the cache is cold. A hash ring moves only about one third of the keys when a server is added or removed. I also place many small points per server on the ring so the load spreads evenly. The trade off is a bit more code, which is worth it for a cache that stays warm when the cluster changes.

### Three cache servers

One server could hold this data with ease. I use three for safety and room to grow. If one goes down I lose one third of the cache and those prefixes just miss to the trie and fill in again, rather than the whole cache going cold at once.

### A short expiry with a little randomness, not clearing on every write

A cached result is a small list of top searches, and most count changes do not change that list. So clearing the cache on every write would be wasted effort. Instead each cached result expires on its own after a short time. I add a little randomness to that time so a burst of saved results does not all expire at the same moment and rush the trie together. The trade off is that a result can be a few seconds stale, which is fine for search suggestions.

### A write log in Redis, then batched writes

Submitting a search must be fast and must not lose the count. So a search is added to a list in Redis and the reply goes out at once. A worker then reads the list in batches, adds up repeats, and writes them in one statement. This turns thousands of writes into a handful, which takes the load off the database. Because the list is saved to disk, a crash does not lose the counts. The worker flushes when the list reaches a set size or after a short time, whichever comes first. The one trade off is that on a crash a batch could be counted twice. For popularity counts that is harmless, while losing counts would not be.

### A sorted set for trending, with fading scores

Trending is a leaderboard that fades. Each batch adds to the score of the searches in it. A sweep every half minute lowers all scores by a set amount, using a half life. So a search that stops being searched slowly drops, while ones that keep coming in stay near the top. This stops a short spike from sitting at the top forever. Reading the top searches is a single fast call to Redis and never touches the database. I kept the set capped to the strongest few hundred so it cannot grow without bound.

### Counts are allowed to be a little behind

The suggestion counts are an estimate of popularity, so being a moment out of date does no harm, while speed and staying up do matter. So I serve from the cache in normal use and let the real count catch up in

the background. The reply to a search is a quick confirmation, not a promise that the database is already updated.

## 6. Performance

Measured on one laptop with the full stack in Docker and 150,000 searches loaded. The numbers come from a script that drives the API with realistic traffic and then reads the metrics endpoint. Run it with `pnpm --filter @ta/server bench`.

| Metric | Result |
|---|---|
| Suggestion latency on the server | p50 0.38 ms  p95 0.65 ms  p99 0.84 ms |
| Cache hit rate (realistic read mix) | 99.4 percent (7948 hits, 52 misses) |
| Database reads to answer a suggestion | 0 |
| Write reduction (20,000 searches) | about 7.6 times fewer rows and about 1800 times fewer writes |
| Cache balance (6000 keys, 3 servers) | 1943 / 1883 / 2174, within about 15 percent of even |
| Building the index (150k searches) | about 350 ms at startup |

The headline is that a suggestion is answered in well under a millisecond and never touches the database, and that a heavy stream of searches lands on the database as a small number of batched writes.

## 7. Running it

```
docker compose up -d                       # postgres on 5433, redis on 7001-7003
cp .env.example .env
pnpm install
pnpm --filter @ta/server load              # load ~150k searches
pnpm --filter @ta/server start             # backend on http://localhost:8080
pnpm --filter @ta/web dev                  # UI on http://localhost:3000
```

Tests for the trie, the ranking, the hash ring, and the metrics run with `pnpm --filter @ta/server test`. The full reasoning behind every choice is in `docs/DESIGN.md` in the repository.